The Foundation
0%

Module 1: QUICK START (...
1.1 What You Need (5 Min Set...
1.2 Your First Folder
1.3 How to Structure Any Pro...

Module 2: WHY THIS WO...
2.1 Video Text Guide (Series ...
2.2 One Line of Python Trigge...
2.3 How a 1953 Word Game E...
2.4 The Ladder That Explains ...
2.5 Clawdbot (Moltbot) Has 1...
2.6 Video as Code: My AI Ani...
2.7 From Nazi Psychology to ...

Module 3: FOLDER ARCHI...

2.4 The Ladder That Explains Every AI Failure

14:26

What You'll Get From This

A framework for figuring out where your work sits on the abstraction ladder, so you stop investing time in things the next model update will hand out for free.

The written version below covers the same ground as the video. The video above walks through the examples with visuals that make the layers click faster. The written version is the one you hand to Claude or come back to for reference.

3.2 Customizing for Your Use ...
3.3 Common Mistakes and H...

A Fortune 500 company spent $200,000 building a custom AI workflow for document processing. Custom features, internal rollout, full team training. Then the platform they were feeding documents into added the same process natively. One API call.

A single button. Months of work became a toggle in someone else's settings menu.

This keeps happening. Startups get wiped by a single feature release. Agencies build automations that the next version of the tool just does on its own. People invest serious energy into things that AI tools will do for free in a month or two.

The pattern underneath all of these failures is the same. They built at the wrong layer of abstraction.

If you watched **Section 2.2**, you saw how each layer of the computing stack abstracts the one below. If you watched **Section 2.3**, you saw how templates and memory hierarchies connect those layers. This lesson takes that same idea and points it at business decisions. Where you invest your time. Where you spend your money. Which layer you build on.

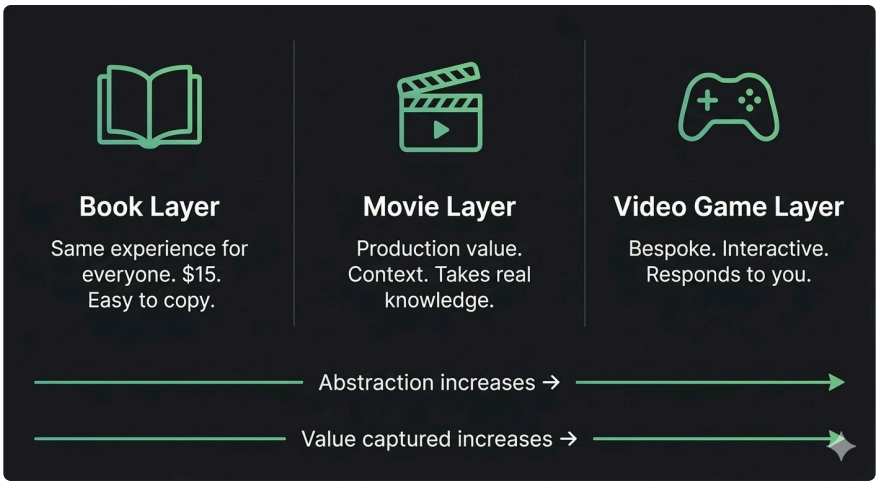
The Framework: Books, Movies, Video Games

Start with a story. As a book, it costs $15. Text on a page. Same experience for every reader. Print a million copies and each one is identical.

Turn that story into a movie. Hundreds of millions of dollars. Actors, visual effects, sound, a production team. Same narrative underneath, but the experience is completely different. It costs exponentially more because the **layer of abstraction** is thicker.

Now turn that story into a video game. GTA 6 is costing billions. But here is what matters: every player's experience is different. You make choices. You control the pacing. The story responds to you. Your playthrough is not the same as mine.

Same story. Three layers. Each one adds more involvement, more control, and each one captures exponentially more value. The experience got harder to replicate, and that is where the



How This Applies to AI and Software

Most people hear "AI can build a website" and think the website is the product. The website was always the book version of something deeper. The raw output. The thing anyone can produce.

The actual value lives in the process underneath. How pieces connect. Which parts need a human. Which parts a tool handles better. Which parts you leave alone entirely.

When I work with a company, the first thing we do is break apart their workflows. Not just engineering. Everything. Go-to-market, HR, content, operations, development. We look at every department's tasks and figure out one by one where AI actually helps, where traditional tooling is better, and where you just need a person. That process alone is far more valuable than people realize.

The Three Layers of AI Use

As you break things apart and start building AI into specific

2

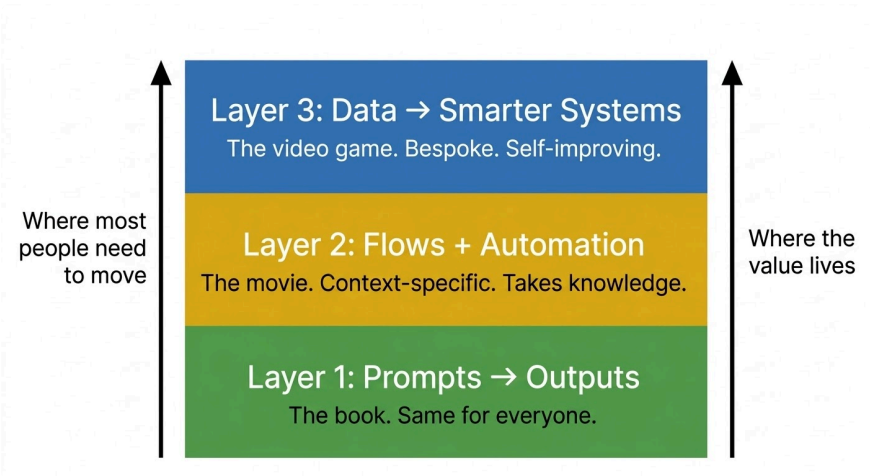
Same tool, same input, same results for everyone. Most people are here.

Layer 2: Building flows. You connect prompts to processes. You understand which tasks in your specific business get automated and which don't. You start learning prompting flows, automation

patterns, connecting tools in ways that are specific to your business, your data, your context. This is the movie layer. Production value. It takes real knowledge to do well.

Layer 3: Systems that learn. The flows you built generate data. What is working, what is not, where the bottlenecks are. You use that data to build better prompts, better agents, better tool calls. The system improves itself because you built it with enough structure to learn from. This is the video game layer. Bespoke, interactive, responsive.

This is the same pattern that happened with programming languages. Machine code to assembly to C to Python. Each layer abstracted the one below into something you could think at a higher level with. The people who thrived were the ones who understood what the new layer made possible.



The Trap

The instinct right now is to automate everything you can. You see a slow process, you throw AI at it, you save time, you feel productive. But few people are asking: is the thing you just automated going to be a built-in feature in 6 months? If it is, you

Don't automate what the next model will give you for free.

Invest in the stuff that is specific to your context. Your domain knowledge, your client relationships, your data. The things that only exist inside your specific situation and can't be scraped and fine-tuned into a general model.

The Web Agency Example

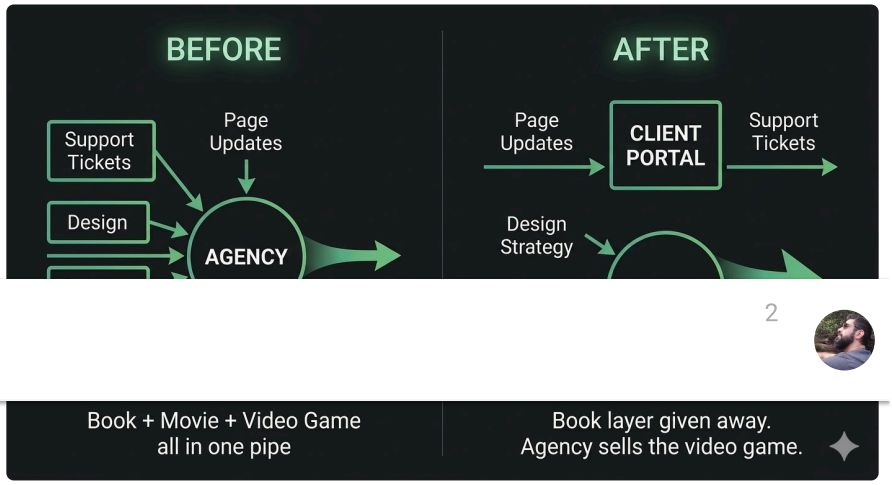
A web agency used to charge $20,000/month to maintain a large client site. Hundreds of pages, multiple teams, constant updates. Can something like Lovable generate pages now? Yeah, for a couple hundred dollars. The layouts, the basic functionality, the raw HTML. That is the book layer. Commoditized.

But this agency doesn't just write HTML. They interview the client's employees to understand internal knowledge structures. They make design decisions based on user psychology and conversion data specific to this company's audience. They know things about the business that you can't prompt because it came from years of being embedded in it.

What they did: stopped fighting the commoditization. Used it. Cut the costs of basic work dramatically. Built a portal where the client's team can handle simple updates themselves. Things that used to need a support ticket and a two-day turnaround.

Now the agency spends almost all their time on the high-context work. Design strategy, research, taste, judgment. The client is involved with the service. They have control over day-to-day. They are playing the game. The agency focuses on the bespoke layer that the game provides.

They gave away the book layer. They charge for the layer above it.



Value Migrates

Every time a new abstraction layer showed up in computing, the layer below didn't die. It became infrastructure. And the new layer created entirely new categories of work that nobody was doing because nobody knew it needed to be done.

When assembly abstracted machine code, the people writing machine code didn't disappear. Some moved up. But new problems showed up that only existed because assembly existed. Problems nobody could have predicted, because you can't see the next layer's problems from the current one. Then C abstracted assembly. Python abstracted C. Frameworks abstracted Python. AI tools are abstracting the frameworks. Every single time, the layer below became a module in the workflow, and the new layer expanded the total amount of work.

Automation doesn't collapse markets. It kills specific positions within markets. The rent-seeking ones, the middlemen, the people who were only in business because friction was protecting them. But the market itself is as durable as human desire.

If AI does your shopping, finds the best price, schedules delivery, handles payment, you are still the one who is hungry. You are still the one who wants a better couch or a specific pair of shoes. The whole transaction chain still ends at a person with a preference.

AI might grow markets more than it shrinks them. Right now, millions of small producers can't reach their ideal customers. The discovery layer is broken. It is all SEO games and ad spend and attention economy middlemen standing between the person who makes the thing and the person who wants it. If AI strips that away and matches need to provider directly, that is a bigger economy. More distributed. More transactions happen.

Look at what you're working on right now. Ask yourself:

- Which layer am I on?
- Is the thing I'm building going to be a feature in someone else's tool soon?

- What do I know, or what access do I have, that only exists in my specific context?

Give away whatever AI made cheap. That is your book layer. That is what gets people in the door. Charge for the layer above.

The question was never "will AI replace me." The question is: which layer am I standing on, and what does the layer above look like?

🔗 **Go Deeper**

- The previous two lessons build the foundation for this one. **Section 2.2** shows what happens between your code and the hardware. **Section 2.3** shows the template pattern and memory hierarchy that connects those layers. This lesson applies both to business decisions. If you skipped either, go back.
- The folder structure from **Section 1.2** is a practical example of building at Layer 2 instead of Layer 1. You are structuring context for your specific work, which is something a general tool can't do for you.
- The full Folder Architecture walkthrough in **Section 3.1** takes this further.

Next Up: 2.5 Clawdbot (Moltbot) Has 100K Stars

You have seen the stack (**2.2**), the pattern connecting the layers (**2.3**), and what happens when you build at the wrong one (2.4). Next: how orchestration and routing work under the hood, and why the model is never the whole story.

🏁 Foundations 2.4 Check-In

You just learned the book, movie, video game framework. Vote below, then tell us in the comments: name one thing in

Poll 138 members have voted

56 62 New comment 6h ago